



A Reinforcement Learning–Inspired Latent Yield-Based Adaptive Algorithm Switching Mechanism

Jayprakash S. Nair¹✉, Jimson Mathew¹, and Shivashankar B. Nair²

¹ Indian Institute of Technology Patna, Patna 801106, Bihar, India
{jayprakash_24a02res06,jimson}@iitp.ac.in

² Indian Institute of Technology Guwahati, Guwahati 781039, Assam, India
sbnair@iitg.ac.in

Abstract. Selecting the most suitable algorithm for a given problem instance remains a challenging task, particularly in online or dynamic environments where problem characteristics evolve over time. Relying solely on instantaneous performance metrics can result in a reactive and unstable behaviour, often leading to suboptimal algorithm switching. This paper introduces a computationally efficient approach for aggregating an algorithm’s performance across multiple problem instances that is fairly immune to erratic variations in instance features. Inspired by features inherent to Reinforcement Learning (RL), this technique encapsulates rewards and penalties into a *latent yield* that, in turn, triggers exploitation and exploration, consequently resulting in adaptive algorithm switching. The proposed technique employs island models, inspired by Genetic Algorithms, to facilitate parallel exploration and performance exchanges among algorithm populations inhabiting local repertoires. Experimental evaluations on sorting algorithms and robotic obstacle-avoidance tasks demonstrate the feasibility and effectiveness of the approach, highlighting its potential in domains where adaptive algorithm selection is critical.

Keywords: Adaptive Algorithm Selection · Island Models · Latent Yield · Intrinsic and Extrinsic Exploration · Reinforcement Learning · Genetic Algorithms · Multi-Agent Reinforcement Learning · Multi-Agent Systems · Robotics

1 Introduction

In the realm of AI, Algorithm Selection (AS) is a hot area that endeavours to choose the near-best algorithm given a set of candidate algorithms that can solve a problem instance [8]. The problem of AS concerns identifying the most appropriate algorithm for solving a given problem instance or a class of problems. A typical AS comprises of a repertoire of algorithms, a problem instance space, a performance measure space and a feature space, all of which play a vital

role in facilitating the switch to another algorithm. Despite the abundance of algorithms for a given computational task, their performance can vary drastically across instances due to factors such as input structure, resource constraints, and problem scale. Selecting an unsuitable algorithm may lead to wasted computation and suboptimal results, whereas a well-chosen algorithm can deliver significant performance improvements in efficiency, accuracy, and robustness. The need for AS arises out of the fact that no single algorithm performs best across all problem instances (No Free Lunch Theorem, NFLT) [23]. This is especially true in cases where the environment is dynamic, where sticking to a pre-decided algorithm may not be worthwhile. AS can be performed either in an offline or online manner. Most of the work is concentrated on the former method. However, if the mapping of the algorithms does not capture the nature of all types of instances, then poor selections are bound to degrade performance. Offline AS methods neglect the information regarding the performance of the selected algorithm on new online instances [3]. AS has been applied in various domains, including data-driven models [15], and multi-armed bandit algorithms [19]. Traditional AS methods assume static problem conditions, but many real-world systems, such as cloud computing and robotics, require online adaptive selection of algorithms. In such systems, the algorithm choice needs to be revised continuously based on feedback from the environment. Reinforcement Learning (RL) and Bayesian optimization have proven useful for such adaptive strategies [10]. Evolutionary computation and Population Based Training (PBT) [5] methods evolve algorithm portfolios, share performance information, and co-adapt algorithmic behaviour to dynamic conditions [7]. In this paper, we propose a novel and computationally simple method for online AS that constantly monitors the performance of the algorithm, accumulates and converts it into a latent yield, which in turn is used to make a decision on whether or not to perform a switch. While this latent yield is synonymous to a combination of rewards and penalties, just as in a typical RL algorithm, the proposed method also inherently performs exploitation and exploration. In addition, the method makes use of parallelism by conforming to the island models used in Genetic Algorithms (GA) which makes it scalable, both in terms of parallelism and an increase in the number of algorithms. The significant contributions of the work described in this paper include –

1. A cumulative latent yield that remembers the overall performance of an algorithm
2. Delayed switching, thereby avoiding a reflex action
3. Parallelism in AS using island models
4. Scalability in terms of an increase in both the number of islands and new algorithms

Subsequent sections describe the main methodology, system dynamics, exploration and exploitation mechanisms, followed by results when the AS was applied to a closed and open world problem.

2 Related Work

Rice [14], formalized the AS problem by mapping problem features to suitable algorithms. Learning-based systems such as Satzilla [24] and ASlib [2] use supervised meta-models for prediction, but remain limited to static or batch settings and are unsuitable for evolving online environments. Recent work has therefore explored adaptive and online AS, with RL and bandit-based methods enabling dynamic switching among algorithms during execution [11]. While effective, these approaches often rely on instantaneous reward feedback, which can cause instability or premature switching when performance fluctuates rapidly. Island model architectures, developed to cater to GAs [21], have also been used to promote diversity in optimization by enabling parallel exploration with periodic migration. Similarly, decentralized and distributed frameworks inspired by biological systems have been proposed for online AS in networked environments. Semwal [18] describes a decentralized Cyber-Physical System (dCPS) mechanism that evolves mappings between problems and solutions across a network of heterogeneous nodes. Our work described in this paper highlights the benefits of distributed knowledge sharing and continuous learning for selecting optimal solutions without relying on central coordination. It positions itself at the intersection of these paradigms—proposing a latent yield-based cumulative estimator for algorithm performance, combined with an island-based exploratory-exploitative architecture, designed to prevent knee-jerk switching.

3 Methodology

In computational science, optimization, and multi-agent systems, island models (also called distributed models or archipelagos) [22] refer to a way of structuring a population of agents in semi-independent groups called islands. A significant challenge in many problems is determining the optimal times to seek improved solutions within the island (exploitation) or from other islands (exploration). The proposed mechanism is designed to address this issue through a multi-agent framework organized as an archipelago, shown in Fig. 1. As can be seen, multiple islands exchange information via a Central Interface Agent (CIA). Each island has its own dedicated agent that can run an algorithm to cater to input task instances. The island also has a repertoire of algorithms and a Yielory that holds a latent yield, which is crucial to AS. We describe this in more detail below.

Let $I = \{I_1, I_2, I_3, \dots, I_n\}$ represents a finite set of n islands, each of which could be considered as a separate computing device capable of hosting an island and its constituents. These islands communicate via a CIA situated in either of these islands or a separate computing device. The CIA does not actively impose decision rules on the agents but primarily observes and keeps track of the best-performing algorithm across the islands based on its performance. Each island $I_i \in \mathcal{I}$ is endowed with an agent ψ_i capable of executing the concerned task for which the system has been designed. In addition, every island has a *repertoire* of

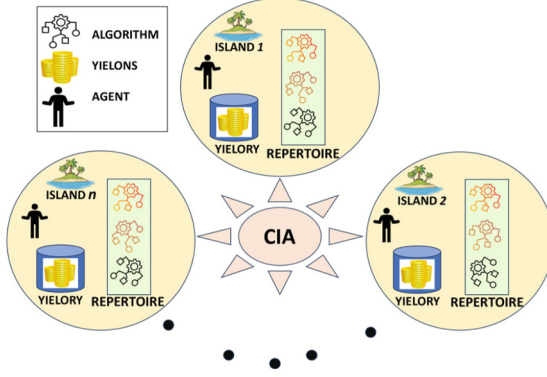


Fig. 1. System architecture for the proposed AS Mechanism.

algorithms, $\text{Rep}_i = \{A_{i1}, A_{i2}, \dots, A_{im_i}\}$, $m_i = |\text{Rep}_i|$, where m_i denotes the number of algorithms associated with the agent ψ_i in I_i . All algorithms within all repertoires are designed to accomplish the same task, T . Agents within all the islands, thus, strive to find the best algorithm for the same task, T . Further, the repertoires are not necessarily disjoint. For distinct agents $\psi_i, \psi_j \in \Psi$, $i \neq j$, it is not necessary that, $\text{Rep}_i \cap \text{Rep}_j \neq \phi$ indicates that repertoires of different islands could have some algorithms in common.

3.1 Agent-Side Actions

Initially, an agent, ψ_i , in an island, I_i , randomly selects an algorithm from its repertoire, $A_{\psi_i} \in \text{Rep}_i$, available within its island. The agent uses this algorithm to perform a series of task instances, $x_j \in \mathcal{X}$, of the same task T , which it encounters. After performing each task instance x_j , the agent ψ_i , receives an extrinsic reward or score, which we term as *credit*. This is proportional to the computational resources R , and time τ , required to perform the task instance using the selected algorithm. Since different agents may execute different algorithms and task instances can also vary in their intrinsic parameters, the credit values could differ across agents. Further, since the algorithms within an agent's repertoire are heterogeneous, the credits earned by each of them on execution by the agent could lie in different scales. To allow comparison across these algorithms in the repertoires, we define a normalized credit as -

$$C_{\psi_i, A_j}^{\text{norm}}(x_j) = \frac{k/f(R_{A_j}, \tau_{A_j}, x_j)}{\max_{x_k \in \mathcal{X}_{A_j}} (k/f(R_{A_j}, \tau_{A_j}, x_k))} \times 100 \quad (1)$$

where $f(C_{A_j}, \tau_{A_j}, x_j)$ is a task-specific cost function, and k is a positive scaling constant, $\mathcal{X}_{A_j}$ denotes the set of all task instances executed by the agent, ψ_i , while using A_j , and x_k refers to the instance of task for which the algorithm performed the best. This normalized credit provides a percentage measure of

performance of the algorithm, which is task-specific and suitable for use in subsequent aggregation steps. It is utilized to compute a latent yield, described in the next section, similar to the one used by Jha [6]. This yield has the potential to generate an innate drive within an agent, which in turn, assists it in determining whether or not to switch from the currently active algorithm to another. Algorithm switches can occur either internally (i.e., to an algorithm available in the repertoire within an island) or externally (i.e., to an algorithm in the repertoire of another island).

3.2 Latent Yield

We coin a term *Yielon* (Υ) to represent a unit for the latent yield produced by an agent after running an algorithm on an instance of a task. The dynamics of Υ mechanism is akin to wringing water from a towel soaked in water. The first wring with a torque q , can extract a high amount of water from the towel that acts as a water repository. To extract the same amount of water once again requires more exertion or a torque higher than q , unless the towel is *charged*, by some means, with more water. The dynamics of the Yielon is based on a similar principle. If the algorithm run by an agent performs well (i.e., when its normalized credit gain is high), the system tends to recharge the Yielon repository or Yielory (Υ_{rep}), proportionate to its current Yielon count, thereby increasing the number of Yielons. If the algorithm performance worsens, it tends to deplete the Yielons in the Yielory proportionate to its current content. The dynamics of this charging and depletion of Yielons is discussed in the following section.

3.3 Algorithm Switching Dynamics

Let $\Upsilon_{curr,i}$ be the Yielons accumulated by an agent ψ_i while running an algorithm $A_i \in Rep_i$. After execution of every instance, x_j , of a task, the normalized credit is computed using the Eq. 1. A set of recent normalized credit values, $C_{\psi_i}^{\text{norm}}(A_i, x_j)$, is maintained using a sliding window of size, w . The mean of the consecutive differences of these values within the window constitutes what is termed as the *Squeezing factor*, σ , calculated as:

$$\sigma = \frac{1}{w-1} \sum_{j=1}^{w-1} (C_{\psi_i}^{\text{norm}}(x_{j+1}) - C_{\psi_i}^{\text{norm}}(x_j)) \quad (2)$$

where:

- w denotes the number of elements maintained in the normalized credit window of size W ,
- $C_{\psi_i}^{\text{norm}}(x_j)$ represents the normalized credit values obtained by agent, ψ_i , on performing the j^{th} task instance x_j ,
- the term $(C_{\psi_i}^{\text{norm}}(x_{j+1}) - C_{\psi_i}^{\text{norm}}(x_j))$ captures the local normalized credit gradient earned from consecutive instance performances.

This squeezing factor, σ , acts as the torque, and regulates the replenishment and depletion of Yielons stored within a *Yielory*, whose contents in turn influence the decision to replace the current algorithm. The update of Yielons in the *Yielory* is performed based on Eq. 3b and 3a, given below.

$$\mathcal{Y}_{new} = \begin{cases} \mathcal{Y}_{curr} + \sigma * (\mathcal{Y}_{curr}/\mathcal{Y}_{max}), & \text{if } \sigma \not\approx 0 \\ \mathcal{Y}_{curr} + p * (\mathcal{Y}_{curr}/\mathcal{Y}_{max}), & \text{if } \sigma \approx 0 \end{cases} \quad (3a)$$

$$(3b)$$

where, $\mathcal{Y}_{curr}$ is the current Yielon count in the *Yielory*, $\mathcal{Y}_{max}$ is its maximum capacity, and p is a small positive value. A higher value of both σ and $\mathcal{Y}_{curr}$ would yield more Yielons to fill up the *Yielory*. From Eq. 3, it is clear that when the current algorithm is faring badly, the extraction of more Yielons would require a substantially higher value of σ , much like the squeezing torque in a physical system. The *Yielory* thus, accumulates lesser Yielons. Conversely, if the *Yielory* was full, even a small amount of squeeze (σ) will cause the extraction of a large amount of Yielons to be added onto the *Yielory*. It may be observed that the first term on the R.H.S. of Eq. 3 indicates the current amount of Yielons in the *Yielory*, while the second form indicates the amount extracted from it. Since the *Yielory* grows or diminishes for every task instance performed, it serves as a sort of memory of how well the algorithm performed over a large period of time. From another perspective, this performance-dependent waxing and waning of the Yielons in the *Yielory* could be looked upon as a consolidated memory of the past performances learned through RL. As mentioned earlier, σ , represents the mean rate of change in normalized credits over recent task instances. A smaller value of σ indicates that the agent's performance is saturating (i.e., the learning process is converging), while a larger magnitude of the same implies an ongoing adaptation or instability in performance. σ is subsequently used to regulate the agent's adaptive AS mechanism. If the *Yielory* had a high number of Yielons, and the algorithm fared badly in the last few instances; the system would still continue to use the currently active algorithm (exploitation), since its Yielon count is rich enough to sustain this fall in performance. On the contrary, if the *Yielory* count is sparse, which means the algorithm has been performing badly for quite some iterations, the depletion rate of Yielons due to a negative σ , is drastic. When this count reaches a lower threshold, it triggers an imminent switch to another algorithm within the local repertoire. The Yielon count thus avoids spontaneous or reflex-based switching of algorithms. This gives the current algorithm more chances to thrive in a given environment.

3.4 Exploitation and Exploration

In search, exploitation, and exploration need to be balanced based on past performances. The content of the *Yielory*, in some way defines the performance of the system over a period of time. Yielons in the *Yielory* grow and diminish based on the performance of the current algorithm. As long as the *Yielory* has a sufficient amount of Yielons to thrive, switching to another algorithm is

inhibited. This means the agent continues to remain satisfied and exploits the currently used algorithm even if its performance is a bit sporadic. However, if the algorithm performs badly for a longer period of time, causing the depletion Yielons beyond a certain lower minimum threshold, the agent is forced to explore and hence switch to a new algorithm in the repertoire of its own island (*intrinsic exploration*) or that from another (*extrinsic exploration*). On the contrary, there could be other conditions when the Yielory gets filled to its maximum and saturates to Υ_{max} due to continuous good performance exhibited by the algorithm, indicating a possible local optimum. The squeezing factor σ , computed using the mean gradient method as presented in Eq. 2, determines whether an agent ψ_i should exploit its currently active algorithm A_{ψ_i} or explore a new one. The conditions for deciding this are given below:

1. When $\sigma \approx 0$, it signifies that the normalized gratuities within the window, W , have saturated. To decide whether to exploit or explore, the difference:

$$\Delta = C_{\psi_i j}^{\text{norm}}(A_{\psi_i j}, x_j) - C_{min}^{\text{norm}} \quad (4)$$

is computed. Here, $C_{\psi_i j}^{\text{norm}}(A_{\psi_i j}, x_j)$ is the normalized credit of the current instance x_j and C_{min}^{norm} is a predefined minimum desirable normalized credit threshold.

- (a) If $\Delta \geq 0$, the agent continues to exploit the current algorithm. In this case, the Yielon count is updated using Eq. 3b, thereby increasing it marginally.
- (b) If $\Delta < 0$, the agent switches to exploration by querying the CIA for the best active algorithm across all islands, A_{best} , and its corresponding Yielon count, Υ_{best} .

If either

$$A_{best} = A_{\psi_i} \quad \vee \quad |\Upsilon_{best} - \Upsilon_{curr}| < \epsilon \quad (5)$$

for some small threshold $\epsilon > 0$, then the agent opts for *intrinsic exploration* by randomly selecting a new algorithm from its repertoire, Rep_i as below:

$$A_{new} \sim \text{Random}(Rep_i \setminus \{A_{\psi_i}\}) \quad (6)$$

Else, it undertakes *extrinsic exploration* by switching to the best algorithm, A_{best} .

2. When the gradient of the normalized gratuities in the window is approximately non-zero, ($\sigma \not\approx 0$) the agent continues to *exploit* the current algorithm. Thus, when the Yielon count increases ($\sigma > 0$) or decreases ($\sigma < 0$) and remains between the Υ_{max} and Υ_{min} , the agent sticks to exploiting the current algorithm as per Eq. 3a.
3. When $\Upsilon_{new} < \Upsilon_{min}$, the agent switches to another algorithm within its repertoire exhibiting *intrinsic exploration*.

The exploitation strategies allow the agent time to improve the performance and avoid any impulsive change of algorithm. Algorithm 1 shows the algorithm for the proposed algorithm switching mechanism.

Algorithm 1 The proposed AS mechanism**Require:** $\text{Rep}_i = \{A_1, A_2, \dots, A_{m_i}\}$

```

1: Initialize:  $\Upsilon_{max}, \Upsilon_{min}, \Upsilon_{curr}, \Upsilon_{initial}$ 
2: Assign an algorithm  $A_j \in \text{Rep}_{\psi_i}$ 
3:  $\Upsilon_{curr} = \Upsilon_{initial}$ 
4: for each instance,  $x_j \in X$  of task  $T$  do
5:   Execute algorithm  $A_j$  on instance  $x_j$ 
6:   Measure computational effort  $R$  and execution time  $\tau$ 
7:   Compute Normalized credit,  $C_{\psi_i, A_j}^{\text{norm}}(x_j)$  (using Eqns. 1)
8:   Compute  $\sigma$  (using Eqn. 2)
9:   if  $\sigma \not\approx 0$  then
10:     Update  $\Upsilon_{new}$  (using Eqn. 3a)
11:   else
12:     Compute  $\Delta$  (using Eqn. 4)
13:     if  $\Delta \geq 0$  then
14:       Update  $\Upsilon_{new}$  by a small threshold  $p$  (using Eqn. 3a)
15:     else
16:       Query CIA for  $A_{best}, \Upsilon_{A_{best}}$ 
17:        $\Upsilon_{curr} = \Upsilon_{initial}$ 
18:       if Condition satisfied (5) then
19:         Select  $A_{new}$  using Intrinsic Exploration (Eqn. 6)
20:       else
21:          $A_{new} \leftarrow A_{best}$  (Extrinsic Exploration)
22:       end if
23:     end if
24:   end if
25:   if  $\Upsilon_{new} < \Upsilon_{min}$  then
26:      $\Upsilon_{curr} = \Upsilon_{initial}$ 
27:     Select  $A_{new}$  using Intrinsic Exploration (Eqn. 6)
28:   end if
29: end for

```

3.5 Galapagos Island

If all agents across the islands were to behave similarly, the overall search process could stagnate around a local optimum, with all islands converging to identical or nearly equivalent algorithms in terms of performance. To prevent such a premature convergence, it is essential that at least one island hosts an agent that exhibits a deviation in exploratory behaviour which is distinct from the rest. This agent, residing on what we term the *G-Island*, (G for Galapagos) actively promotes diversity across the islands, by evaluating comparatively unexplored or underutilized algorithms. When the agent within *G-Island*, I_G , initiates exploration, the new algorithm, A_{new} , is selected based on the conditions outlined below.

$$\begin{aligned}
 (I = I_G) \wedge & \left[(\Upsilon_{\text{mean}} - \Upsilon_{I_G} > \delta) \right. \\
 & \left. \vee \left(I_G = \arg \max_{I_i \in \mathcal{I}} \text{Perf}(I_i) \wedge \exists I_j \neq I_G : A_j = A_G \right) \right] \\
 \Rightarrow A_{\text{new}} = & \text{Random} \left(\text{Rep}_G \setminus \{A_i \mid I_i \in \mathcal{I}\} \right)
 \end{aligned} \tag{7}$$

It may be noted that the first condition indicates that the Yielories of all islands have reached their maximum capacities. The second ensures that it selects an algorithm from within its internal repertoire, which is different from the currently active algorithms used across the islands.

4 Experimental Setup

Experiments were performed using two different domains and a randomly assigned, I_2 as the G-Island, with the following parameter values: $\Upsilon_{\text{max}} = 100$, $\Upsilon_{\text{min}} = 30$, $\Upsilon_{\text{initial}} = 60$, $W = 5$, $p = 0.05$, $\Delta = 80$, and $\epsilon = 10$.

4.1 Finding the Best Sorting Algorithm

To test the proposed method, we utilized three sorting algorithms - Quick-sort (QS), Insertion Sort (IS), and Counting Sort (CS) - distributed across the repertoires of three distinct islands. The repertoires on each island contained these three sorting algorithms. Each island had its own randomizer function, which generated different data types of length 20 and fed them to the active sorting algorithm. The randomizers generated three types of input data at random:

1. **Randomized Large (RandL)**: An array of random numbers having a large range $(0, 10^6)$, ideally suited for Quick-sort.
2. **Randomized Small (RandS)**: An array of random numbers with a small range (1 to 35), suited for Counting Sort.
3. **Almost Sorted (AlmoS)**: An array of nearly sorted numbers, perfect for Insertion Sort.

The randomizers ensured that each active algorithm received different combinations of these input types. This allowed for a comprehensive evaluation of how each sorting algorithm performs with varying types of data and conditions, enabling the testing of efficiency and suitability of each algorithm for different types of input data or sorting instances. After each array was sorted, the associated credits were calculated using the following equation:

$$C_{\psi_i, A_j}(x_j) = \text{round} \left(\frac{1}{\omega + \alpha + 1} \right) \tag{8}$$

where ω is the number of instructions executed, and α represents the number of memory accesses. These credits were used to compute the normalized credits

and subsequently the Yielon count, which in turn aids in switching the algorithms. For comparison, baseline experiments were conducted without using the Yielory and G-Island concept. The agents in the islands were initialized with a different algorithm and used a greedy AS strategy. After every episode, this greedy strategy explored the current best algorithm in the archipelago and made the agent switch to it.

4.2 Learning to Avoid Obstacles

This experiment was carried out in the robotics domain to validate the working of the proposed method in near real-world scenarios. Here, the agents were substituted by identical robots inhabiting a simulated setup using Webots [12]. Four instantiations of Webots, each containing a robot, a Yielory, and a repertoire, formed the four islands. Each robot inhabited a different environment with a variable number of static obstacles positioned at random locations. A snapshot of such an environment is shown in Fig. 2. The main objective was to arrive at the right algorithm to avoid the obstacles and sustain within the given environment. We have used three learning algorithms - Q-Learning (QL) [20], SARSA [16], and Double-Q-Learning (DQL) [4]. Further, by conferring two different values of learning rate ($\alpha = 0.1$ and $\alpha = 0.7$) to each algorithm, we created a population of six algorithms in the repertoires. The robots, together with the RL algorithms, constituted a Multi-Agent Reinforcement Learning (MARL) system. If the robot (agent) during its motion does not encounter an obstacle for a certain period of time (5 s), then the active algorithm used by it was conferred a Normalized credit of 10 units. In addition, the charge on the battery of each robot at start was kept at 100% (maximum charge), which, on every action or movement made by the robot, decreased by 2.5% of the maximum. When the charge depletes to zero, an episode that constitutes an instance of learning the task was assumed to have been completed. At the end of each such episode, the Yielory is updated and a new episode commences with the battery once again charged to the maximum.

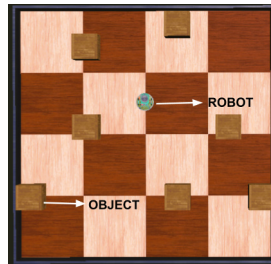


Fig. 2. Snapshot of the environment of one of the Webots' [12] instantiation arena featuring a robot amongst multiple static obstacles at different positions.

5 Results

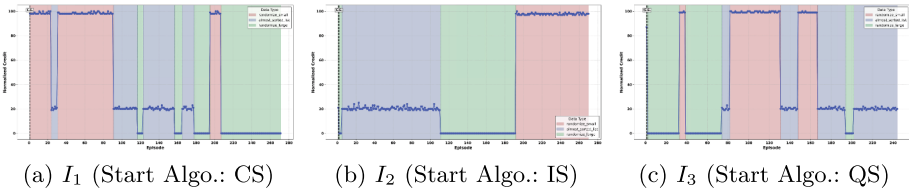
In this section, we discuss the results based on the types of experiments conducted.

5.1 Sorting

Figures 3a–3c, depict the baseline graphs of variations in normalized credits versus episodes, across the three islands when a greedy approach coupled without and with a G-Island was used. The vertical black dashed lines indicate a switch to a new algorithm, mentioned alongside. The vertical colour bands indicate the type of data input to the sorting algorithm during the episodes. In the graph in Fig. 3a, the agent in I_1 started off using CS. Since this algorithm is best suited for data of type RandS, it yielded a high value of normalized credit. I_2 started with IS, which is not compatible to RandL. It thus switched to CS, the best performing algorithm across all islands at that episode (Episode-2). A similar pattern can be observed at I_3 , which also switched to CS (Episode-2). Eventually, all islands switched to CS and stuck to it in spite of varying data types. The islands thus quickly stabilized to the same algorithm, curbing exploration of potentially better algorithms, thereby resulting in a local optimum. In Figs. 3d–3f, the G-Island switched frequently causing the other islands to copy its algorithm every time it was a better one. Subsequently, this caused other islands to also switch to the better one, at about the same rate as the G-Island. Thus, the introduction of the G-Island did not really do much to improve the search for the best algorithm or the number of switches. Figure 4a–4c depict the variations in normalized credits and Yielon count with episodes. It can be seen in Fig. 4a that even though there were erratic changes in data types in I_1 , the presence of a substantial amount of Yielons generated an inertia that delayed switching to a new algorithm from the initial CS algorithm. It later switched to IS at the 100th episode when the Yielon counted dipped to its lowest, retained this algorithm since it was compatible to the data type (AlmoS) till the 200th episode when the data type changed to RandL and then switched many times to find a better performing algorithm. Similar behaviours can be observed in the other two islands (Figs. 4b and 4c) where the Yielon count controls the switching. Figures 4d–4f depict similar graphs when the G-Island was introduced. It can be clearly seen that in the G-island the number switches are far more than the others indicating that it has struggled to keep exploring more often than the other two islands.

Table 1 provides a comparison of normalized credits accumulated at the islands together with the number of switches they made during the runs. From the first two entries, (greedy approaches - with and without the G-Island) it can be observed that the introduction of the G-Island (which causes a huge amount of switching) does not contribute much to increase in Normalized credits. On the contrary, the introduction of the Yielory and the G-Island (3rd and 4th entries) show a substantial increase in the number of Normalized credits accumulated. It may also be noted that the introduction of the G-Island increased

No Yielory and No G-Island



No Yielory and With G-Island

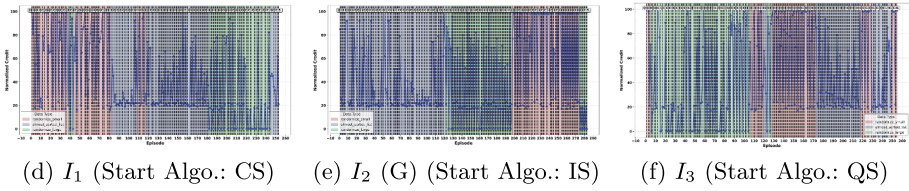
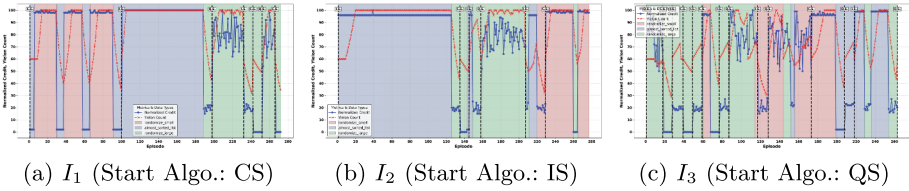


Fig. 3. Variations in Normalized Credits versus Episodes in Islands for the Sorting scenario without and with the G-Island. The vertical dashed lines indicate switches to a new algorithm while the vertical coloured columns indicate the type of input data. (G) indicates the G-Island.

With Yielory and No G-Island



With Yielory and G-Island

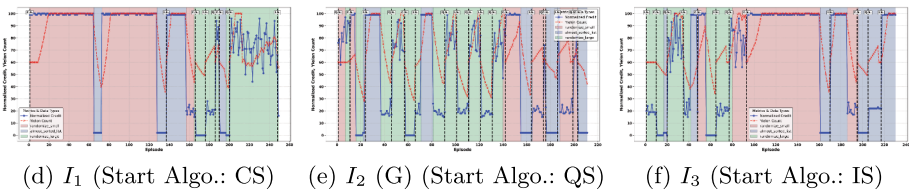


Fig. 4. Variations in Normalized Credits & Yielon Counts versus Episodes in Islands for the Sorting Scenario without and with the G-Island. The vertical dashed lines indicate switches to a new algorithm while the vertical coloured columns indicate the type of input data. (G) indicates the G-Island.

Table 1. Comparison of Normalized Credits & Switches across Islands

S. No.	Experiments Using Sorting	I_1		I_2 (G)		I_3	
		Credits	Switches	Credits	Switches	Credits	Switches
1	No Yielory and No G-Island	10384	0	3058	1	8491	1
2	No Yielory and With G-Island	9569	160	7241	163	7411	163
3	With Yielory and No G-Island	16019	2	15987	3	11102	14
4	With Yielory and G-Island	14469	6	9672	17	12663	11

the number of switches at a marginal cost of Normalized credits in one of the islands (I_1), while for another it improved the same. The take-home is that the Yielory and the G-Island together, drastically decreased the number of switches when compared to those in the greedy approach while also ensuring a balance between exploitation and exploration. It is therefore clear that optimizing the ratio of the number of G-Islands to that of other islands can offer greater insights and improve performance.

5.2 Obstacle Avoidance

The graphs in Figs. 5a–5d depict variations in normalized credits and Yielon count at each episode in the four islands. As can be seen in all the graphs

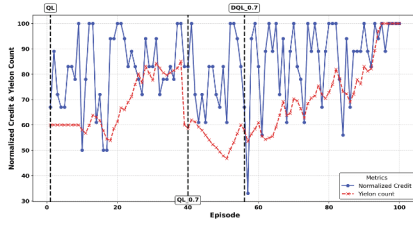
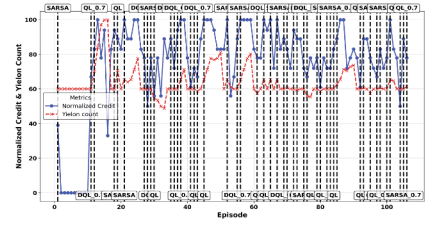
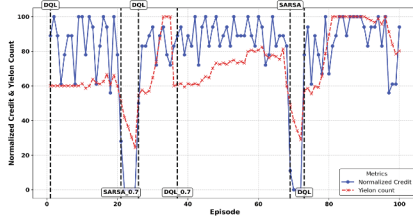
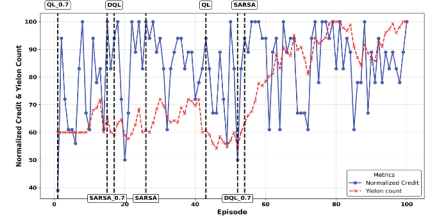
(a) I_1 (Start Algo.: Q-Learning)(b) I_2 (G) (Start Algo.: SARSA)(c) I_3 (Start Algo.: Double Q-Learning)(d) I_4 (Start Algo.: Q-Learning, $\alpha = 0.7$)

Fig. 5. Graphs depicting variations in Normalized Credits and Yielon Counts over Episodes for the Obstacle Avoidance Robotics Scenario in the four islands. (G) indicates the G-Island.

except to an extent in Fig. 5b, algorithm-switches allow the learning to absorb variations in input in spite of sudden changes in the normalized credits. While using learning algorithms that make use of models or policies, as in this case, it is essential that they be given sufficient opportunities to sample and learn from the input before deeming them to be non-performing. As expected, the three islands - 1, 3, and 4 - switch comparatively lesser number of times (Figs. 5a, 5c, and 5d) when compared to the more active G-Island (Fig. 5b). Together, these four islands ensure that saturation does not occur too soon and thus avoid local optima. As a rudimentary ablation study exercise to understand the significance of the G-Island, we removed the same and in lieu, placed a normal island, making all the islands in the archipelago homogenous. It was found that in both the cases - Sorting and Obstacle avoidance - the number of extrinsic explorations reduced, intuitively suggesting that the search seemed to have been curtailed to the algorithms in the respective local repertoires.

6 Conclusions and Future Work

This paper describes an adaptive AS technique inspired by both a natural process and RL, which involves agents working in both isolation as well as in co-operation. Agents within islands containing a repertoire of algorithms, concurrently search for the best possible algorithm for task instances streaming in, thereby exploiting parallelism and hastening convergence. While the agents exploit the current algorithm as long as there are sufficient Yielons in the Yielory, they switch to exploring algorithms within their internal repertoires (intrinsic exploration) when the Yielon counts go below a threshold. Over time, when the Yielon count stabilizes, the agent is made to feel the occurrence of a local optima, forcing it to explore algorithms across islands and import the one that is the best, thereby exhibiting extrinsic exploration. The sharing of algorithms across islands in such extrinsic explorations, accelerated by a hyperactive G-Island, hints at genetic exchanges, which in turn mitigates the occurrence of evolutionary stagnation and premature convergence to suboptimal solutions as in the Island syndrome [1]. The content in the Yielory thus, controls exploitation and exploration during the overall search for the best algorithm and facilitates life-long learning. Results obtained from experiments carried out in the closed world (Sorting) and in an open world (Robotics) clearly demonstrate the efficacy of the proposed technique. With more computing power, an island could be populated with multiple cooperative agents that feed on algorithms in the local repertoire, thereby speeding up convergence. Flushing ineffective or less performing algorithms across the repertoires by maintaining the best in local Halls-of-Fame, as described in [9], will also aid in hastening convergence. Further, the proposed system could be used for life-long learning if new algorithms for either same or heterogeneous tasks were to be added to the repertoires on-the-fly using mobile agents, as described in [17]. The agents in the islands can thus discover improved algorithms capable of tackling multiple tasks. In addition to inter-island algorithm exchanges during extrinsic explorations, hyperparameters

may also be crossed over or mutated, following PBT principles, to ameliorate the performance of an active algorithm. In future we intend to scale up the number of islands and integrate Federated Learning (FL) using mobile agents [13] that migrate across islands to identify high-performing models, aggregate them, and replant their modified versions into island repertoires. This will eliminate the need for a CIA and yield a fully decentralized framework where Island GA, PBT and FL operate in unision. In multi-mobile-robot systems, such a framework will then enable robots to emulate mobile islands whose network topology dynamically evolves with physical mobility.

References

1. Baeckens, S., Van Damme, R.: The island syndrome. *Curr. Biol.* **30**(8), R338–R339 (2020). <https://doi.org/10.1016/j.cub.2020.03.029>
2. Bischl, B., et al.: Aslib: a benchmark library for algorithm selection. *Artif. Intell.* **237**, 41–58 (2016). <https://doi.org/10.1016/j.artint.2016.05.001>
3. Degroote, H.: Online algorithm selection. In: *IJCAI*, pp. 5173–5174 (2017)
4. Hasselt, H.: Double q-learning. In: Lafferty, J., Williams, C., Shawe-Taylor, J., Zemel, R., Culotta, A. (eds.) *Advances in Neural Information Processing Systems*, vol. 23. Curran Associates, Inc. (2010)
5. Jaderberg, M., et al.: Population based training of neural networks (2017). <https://arxiv.org/abs/1711.09846>
6. Jha, S.S., Godfrey, W.W., Nair, S.B.: Stigmergy-based synchronization of a sequence of tasks in a network of asynchronous nodes. *Cybern. Syst.* **45**(5), 373–406 (2014)
7. Kerschke, P., Hoos, H.H., Neumann, F., Trautmann, H.: Automated algorithm selection: survey and perspectives. *Evol. Comput.* **27**(1), 3–45 (2019)
8. Kotthoff, L.: Algorithm selection for combinatorial search problems: a survey. In: Bessiere, C., De Raedt, L., Kotthoff, L., Nijssen, S., O’Sullivan, B., Pedreschi, D. (eds.) *Data Mining and Constraint Programming. LNCS (LNAI)*, vol. 10101, pp. 149–190. Springer, Cham (2016). https://doi.org/10.1007/978-3-319-50137-6_7
9. Kulkarni, D.D., Nair, S.B.: Mutational puissance assisted neuroevolution. In: *Proceedings of the 2020 Genetic and Evolutionary Computation Conference Companion, GECCO 2020*, pp. 1841–1848. Association for Computing Machinery, New York, NY, USA (2020). <https://doi.org/10.1145/3377929.3398149>
10. Lindauer, M., Hoos, H.H., Hutter, F., Schaub, T.: Autofolio: an automatically configured algorithm selector. *J. Artif. Intell. Res.* **53**, 745–778 (2015)
11. Lindauer, M., Hutter, F.: Algorithm configuration and selection on heterogeneous resources. In: *Proceedings of the 27th International Joint Conference on Artificial Intelligence (IJCAI)*, pp. 5178–5182 (2018)
12. Michel, O.: Webots: professional mobile robot simulation. *J. Adv. Robot. Syst.* **1**(1), 39–42 (2004). <http://www.ars-journal.com/International-Journal-of-Advanced-Robotic-Systems/Volume-1/39-42.pdf>
13. Nair, J.S., Kulkarni, D.D., Joshi, A., Suresh, S.: On decentralizing federated reinforcement learning in multi-robot scenarios. In: *2022 7th South-East Europe Design Automation, Computer Engineering, Computer Networks and Social Media Conference (SEEDA-CECNSM)*, pp. 1–8 (2022). <https://doi.org/10.1109/SEEDA-CECNSM57760.2022.9932985>

14. Rice, J.R.: The algorithm selection problem. *Adv. Comput.* **15**, 65–118 (1976)
15. Rook, J., Renau, Q., Trautmann, H., Hart, E.: Efficient online automated algorithm selection in the face of data-drift in optimisation problem instances. In: *Proceedings of the 18th ACM/SIGEVO Conference on Foundations of Genetic Algorithms*, pp. 262–272 (2025)
16. Rummery, G.A., Niranjan, M.: *On-line Q-learning using connectionist systems*, vol. 37. University of Cambridge, Department of Engineering Cambridge, UK (1994)
17. Semwal, T., Bode, M., Singh, V., Jha, S.S., Nair, S.B.: Tartarus: a multi-agent platform for integrating cyber-physical systems and robots. In: *Proceedings of the 2015 Conference on Advances In Robotics, AIR 2015*. Association for Computing Machinery, New York, NY, USA (2015). <https://doi.org/10.1145/2783449.2783469>
18. Semwal, T., Nair, S.B.: A decentralized and distributed immune-inspired mechanism for adaptive problem–solution mapping in cyber–physical systems. *Appl. Soft Comput.* **86**, 105920 (2020). <https://doi.org/10.1016/j.asoc.2019.105920>
19. Tornede, A., Bengs, V., Hüllermeier, E.: Machine learning for online algorithm selection under censored feedback. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 36, pp. 10370–10380 (2022)
20. Watkins, C.J., Dayan, P.: Q-learning. *Mach. Learn.* **8**(3), 279–292 (1992)
21. Stuber, J.: Theory path orderings. In: Narendran, P., Rusinowitch, M. (eds.) *RTA 1999*. LNCS, vol. 1631, pp. 148–162. Springer, Heidelberg (1999). https://doi.org/10.1007/3-540-48685-2_12
22. Whitley, D., Rana, S., Heckendorn, R.B.: Island model genetic algorithms and linearly separable problems. In: *AISB International Workshop on Evolutionary Computing*, pp. 109–125. Springer, Cham (1997)
23. Wolpert, D.H., Macready, W.G.: No free lunch theorems for optimization. *IEEE Trans. Evol. Comput.* **1**(1), 67–82 (2002)
24. Xu, L., Hutter, F., Hoos, H.H., Leyton-Brown, K.: Satzilla: portfolio-based algorithm selection for sat. *J. Artif. Intell. Res.* **32**, 565–606 (2008). <https://doi.org/10.1613/jair.2470>